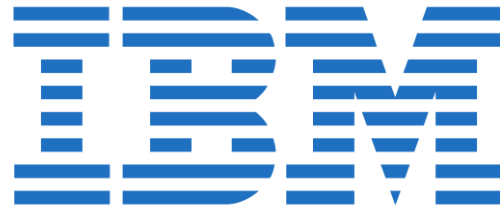


WUKONG

A Scalable and Locality-Enhanced Framework for Serverless Parallel Computing

Benjamin Carver, Jingyuan Zhang, Ao Wang,
Ali Anwar, Panruo Wu, Yue Cheng



Python Data Analytics



DASK



pandas



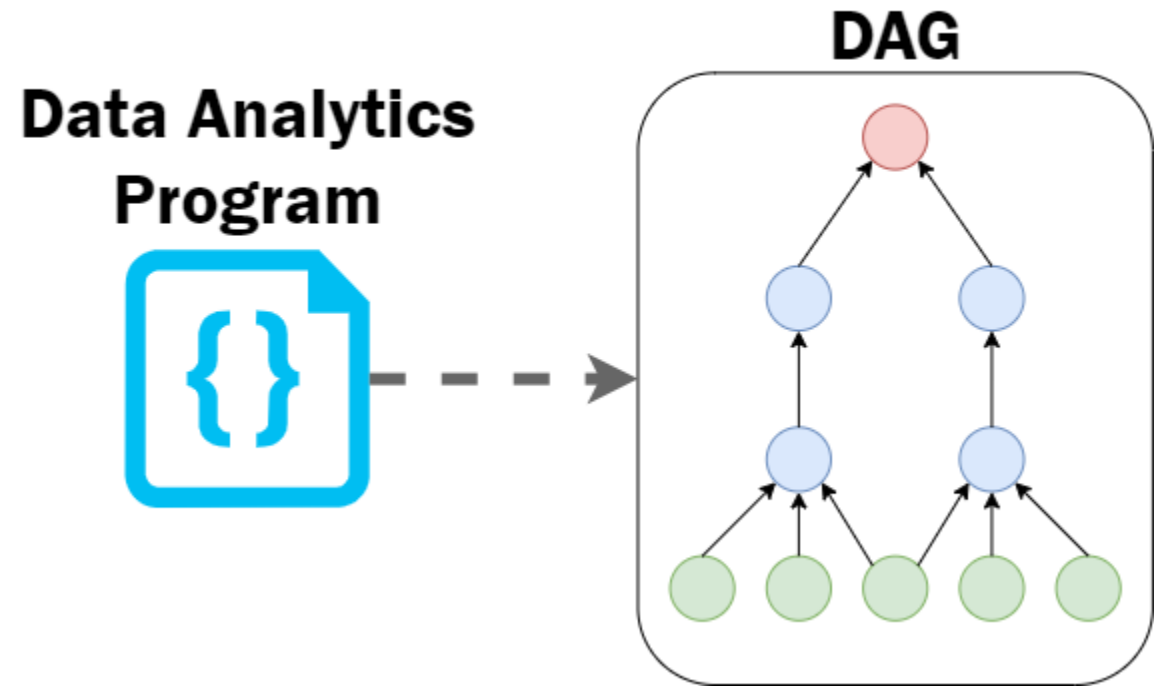
NumPy



SciPy

Parallelizable Large-Scale Python Analytics Jobs

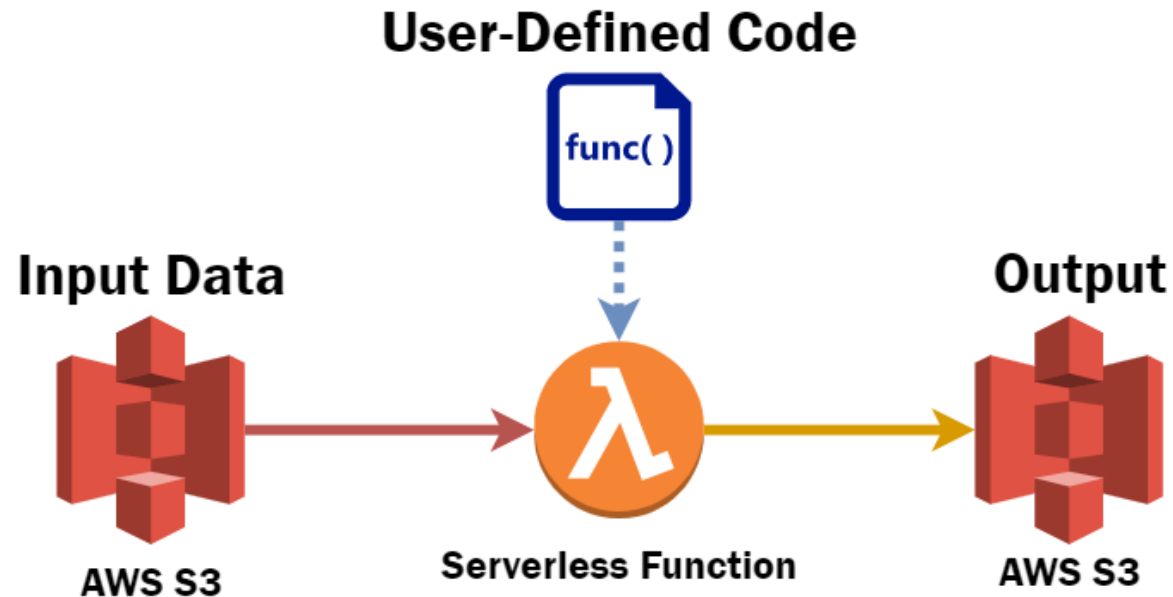
- Some compute- and data-intensive applications are parallelizable.
- Such applications can be modeled as a directed acyclic graph (DAG).
 - Nodes are short, fine-grained tasks and edges indicate data dependencies between tasks.



Serverless Computing

Serverless computing is an emerging **cloud computing** model that enables a new way of **building and scaling** applications and services.

Developers can break traditionally monolithic server-based applications into **finger-grained** cloud functions.



Benefits of Serverless Computing



Cost Effective



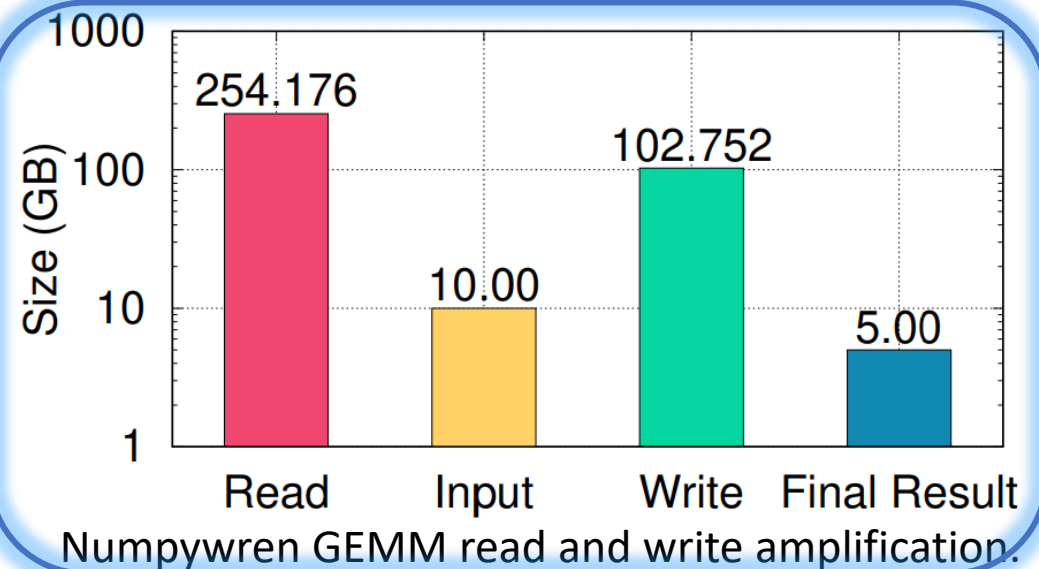
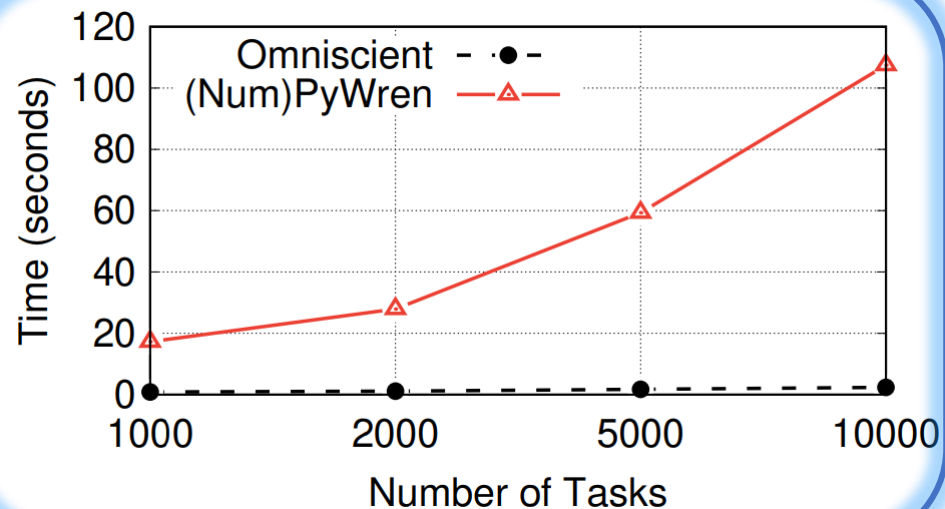
High Elasticity



**Automatically
Provisioned &
Managed**

Challenges to Scale Parallel Programs on Serverless Platforms

1. Challenge to **rapidly scale** to thousands of serverless (Lambda) functions instantly.
2. Challenge of **excessive data movement** due to inherent constraint of serverless platforms (serverless computing restricts communication due to outbound-only network connectivity)
3. Per function **resource limitations** (1 or 2 CPU cores, up to 3GB of main memory, etc.)

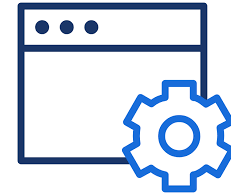


Naively porting a serverful parallel computing framework to a serverless computing platform won't work.

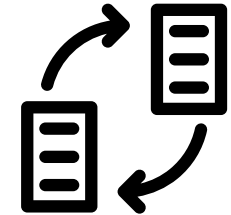
A serverless parallel computing framework may not care about traditional metrics, such as load balancing and cluster utilization.

Key Design Goals

1. Accelerate task scheduling.



2. Reduce data movement.



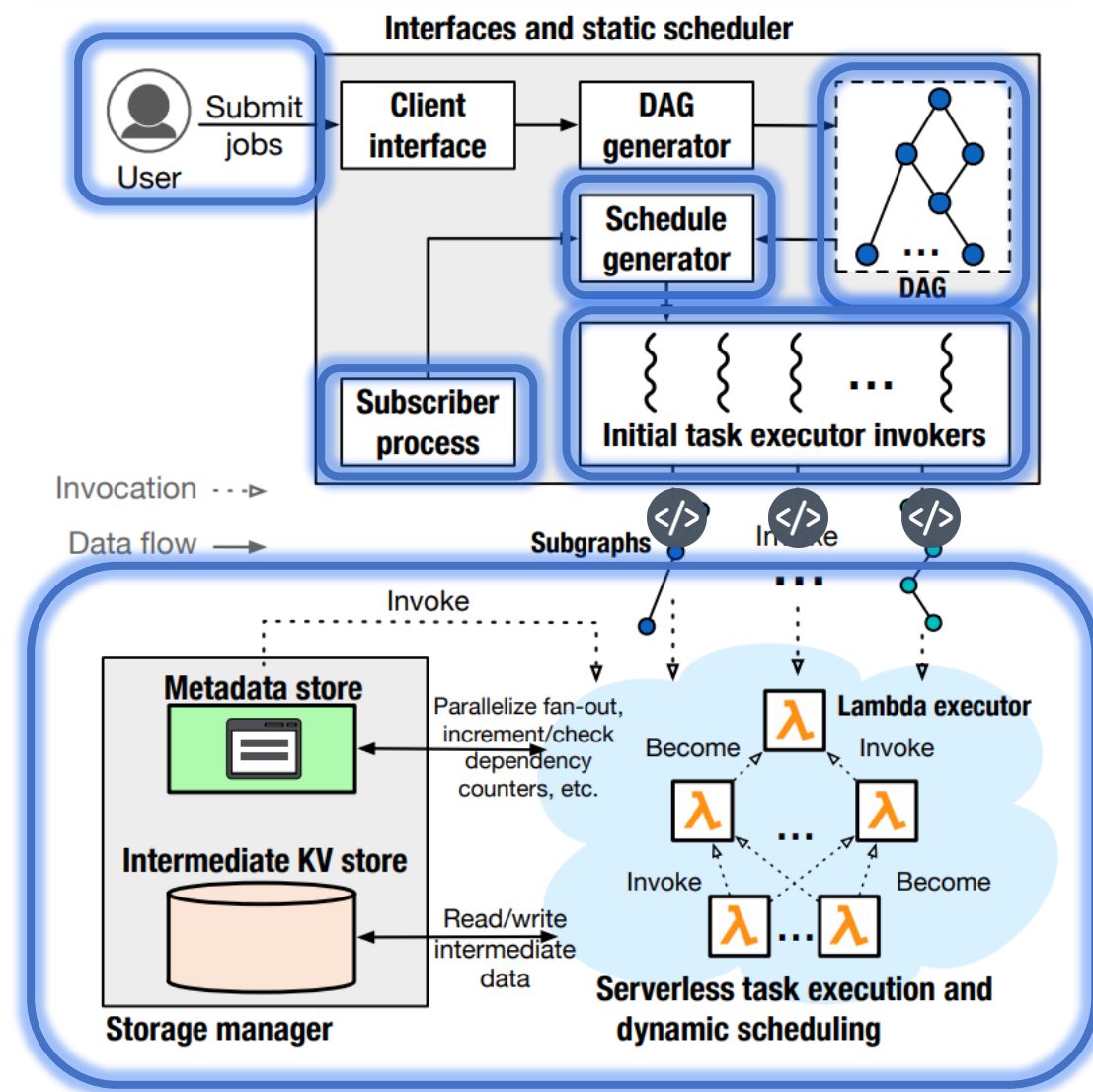
3. Improve cost effectiveness.



Enter WUKONG

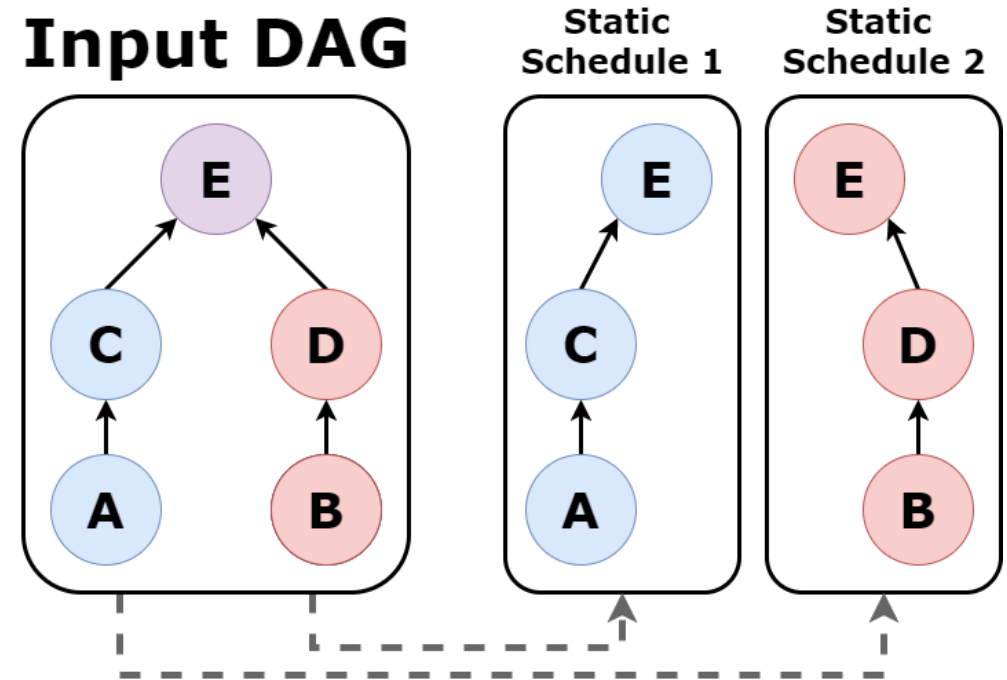
- **WUKONG** is a **serverless-oriented, locality-aware** parallel computing framework.
- **WUKONG** **partitions** the work of a centralized scheduler across many serverless executors, **greatly improving performance**.
- **WUKONG's** design enables tasks to be scheduled and executed **concurrently**, reducing resource contention while providing **automatic resource elasticity** and **cost-effectiveness**.

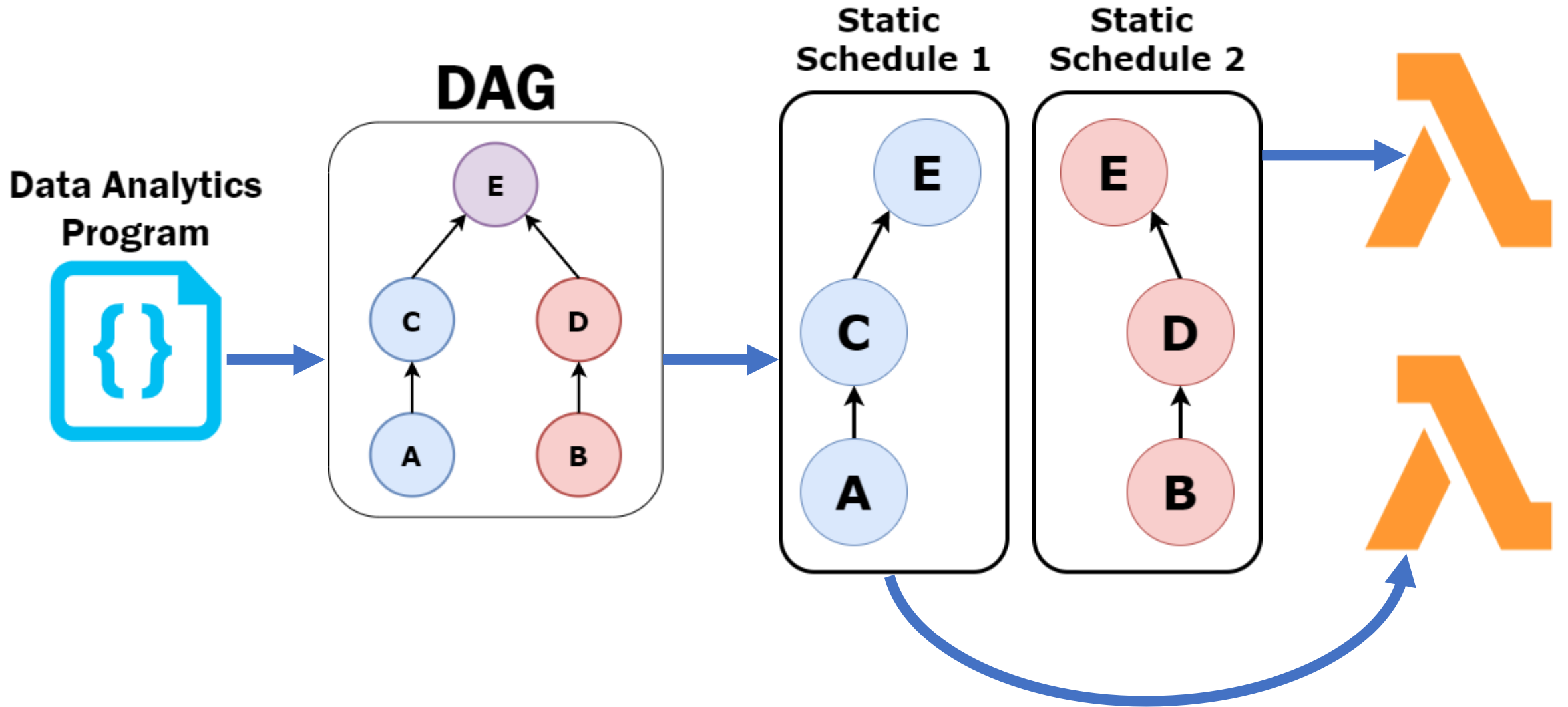
WUKONG's Architecture



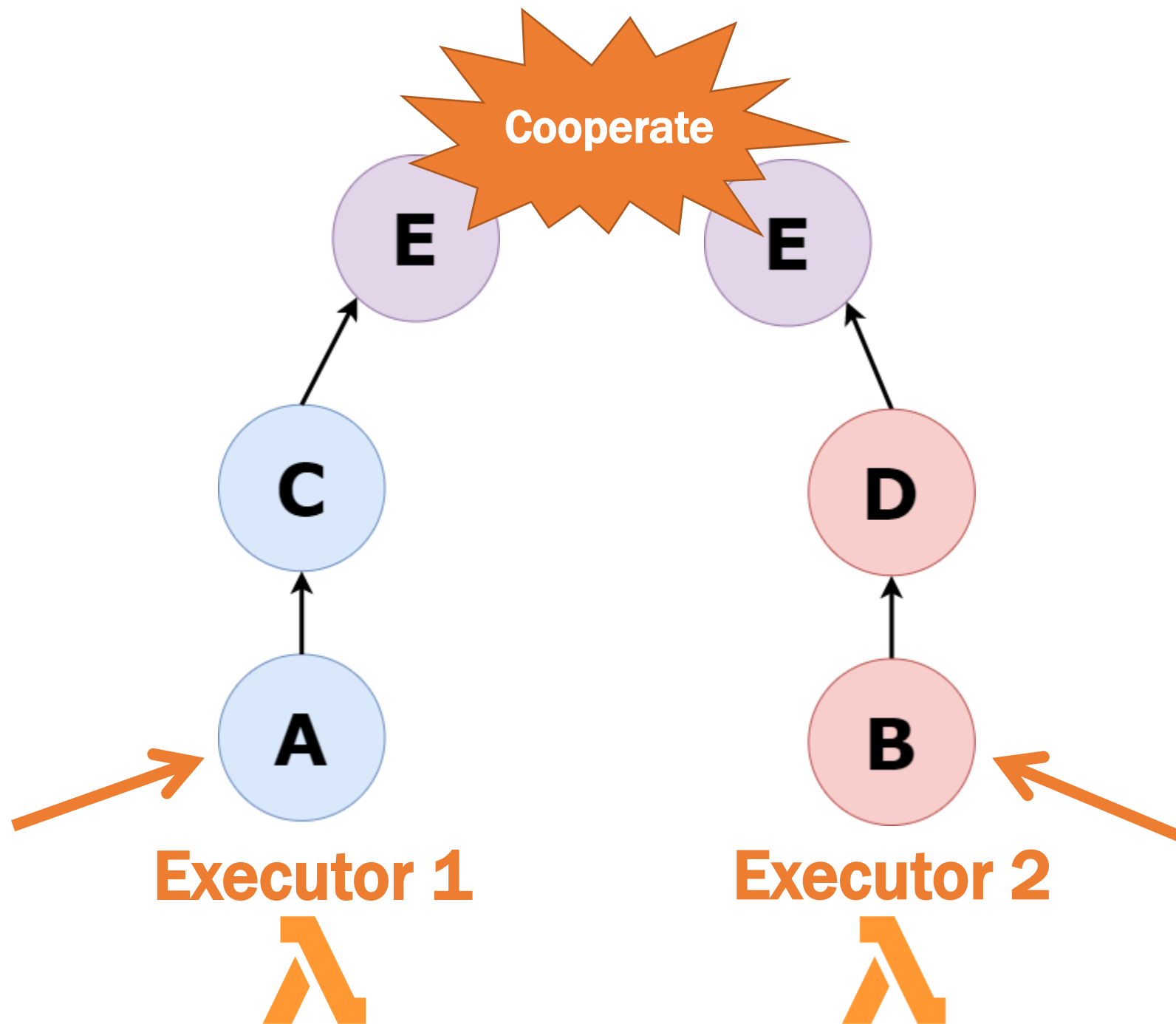
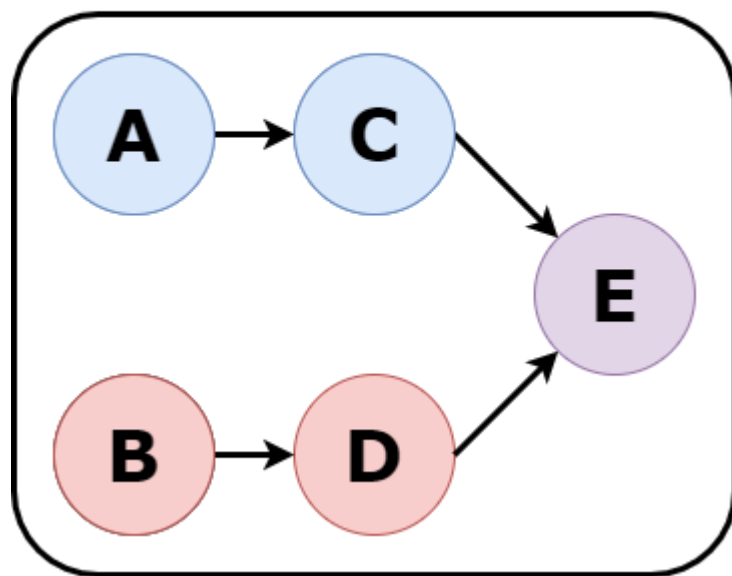
Scheduling in WUKONG

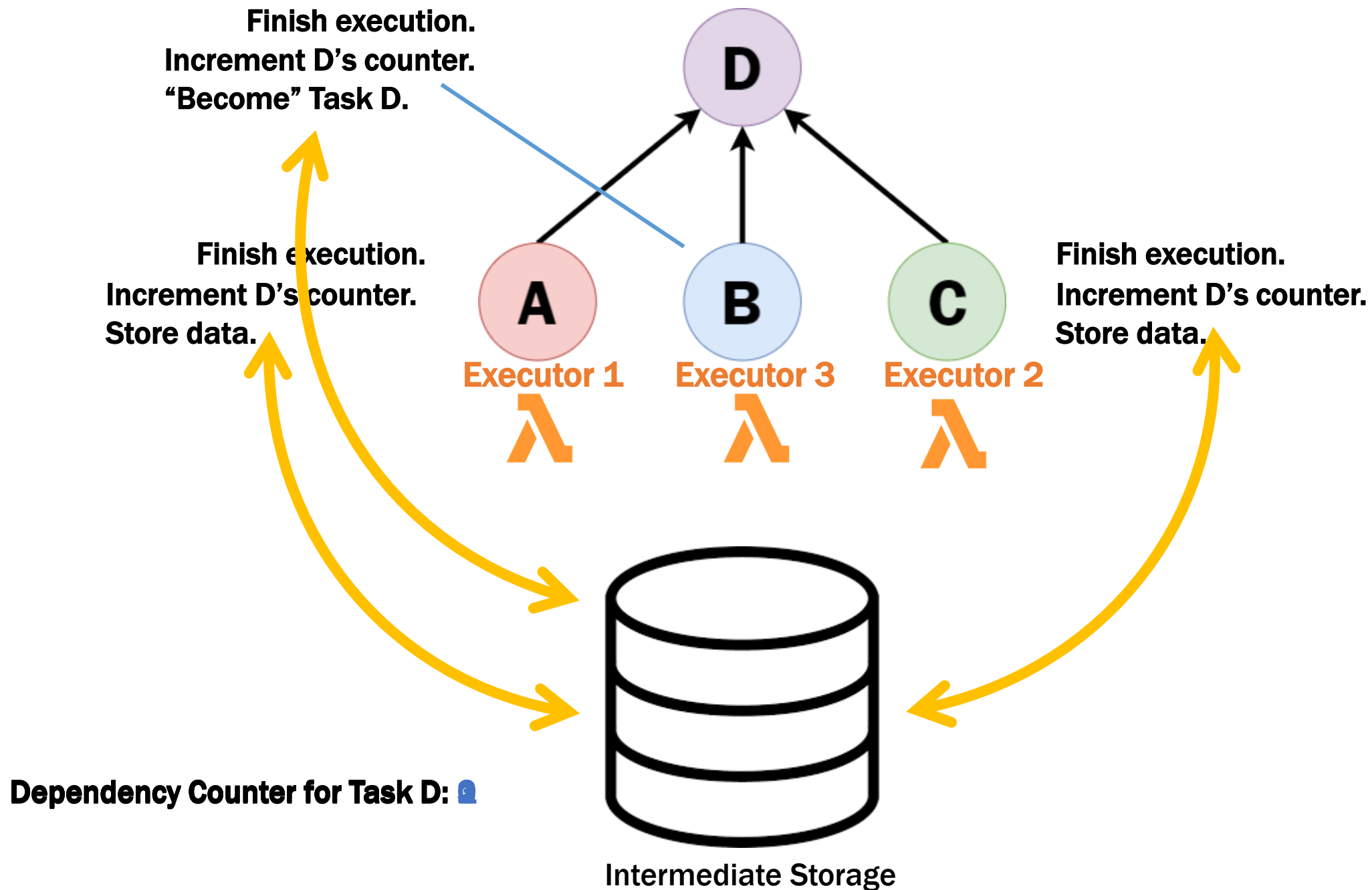
- Combination of **static** and **dynamic scheduling**.
- Input DAG partitioned into **static schedules**, or subgraphs of the original DAG.
- Serverless Executors are assigned a **static schedule**.
- Executors use **dynamic scheduling** to enforce data dependencies and **cooperatively** schedule tasks found in multiple static schedules.

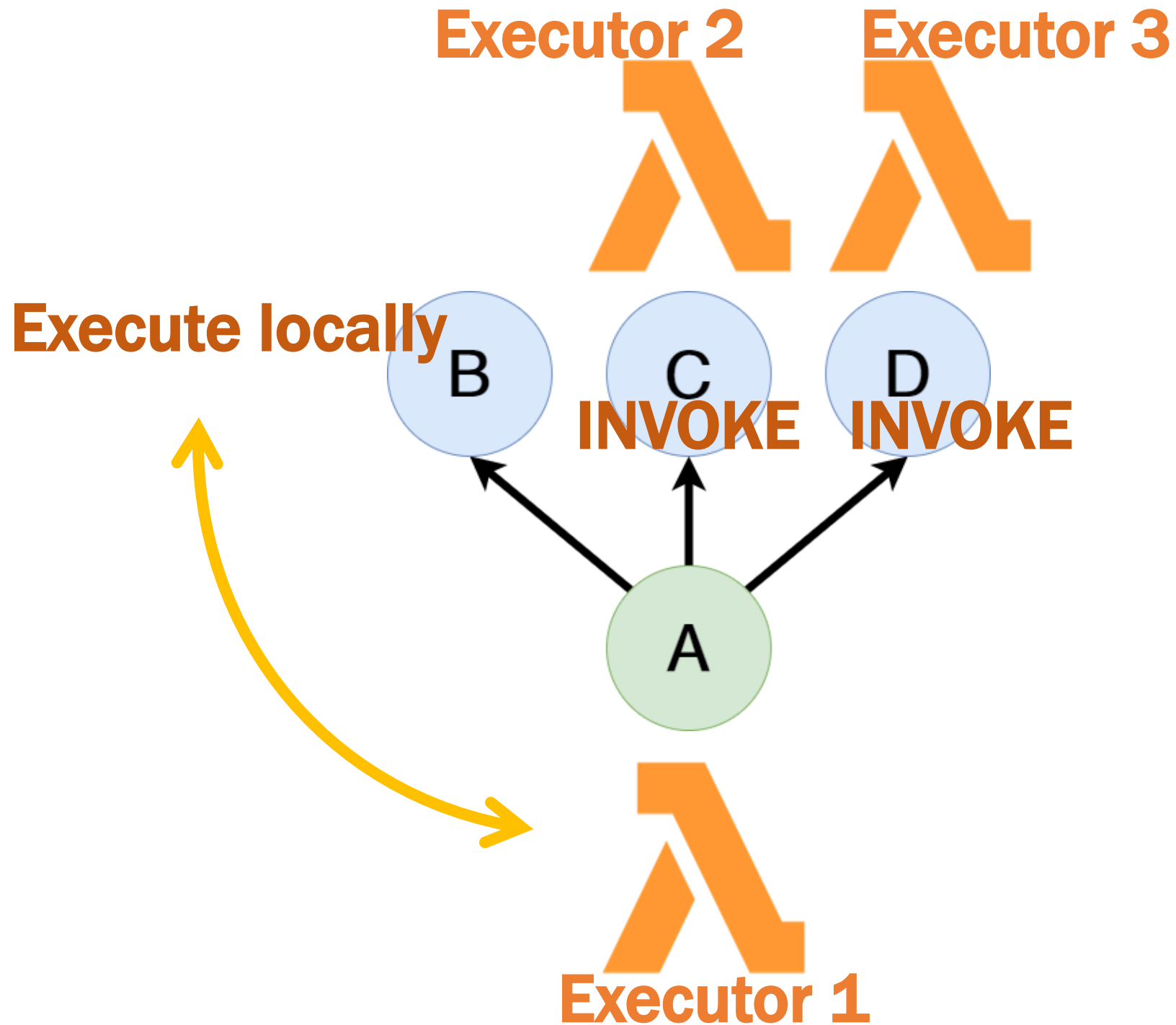




Input DAG







Other Optimizations in WUKONG

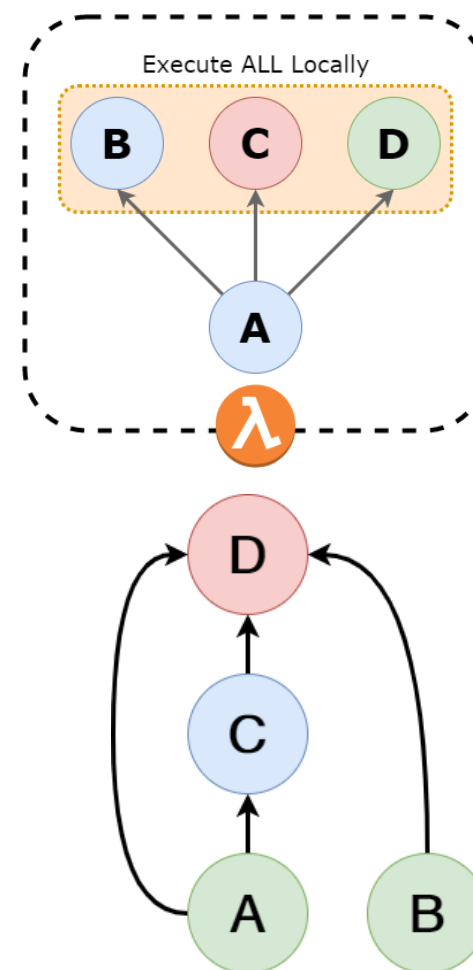
- WUKONG uses several techniques to enhance data locality:

- **Task Clustering**

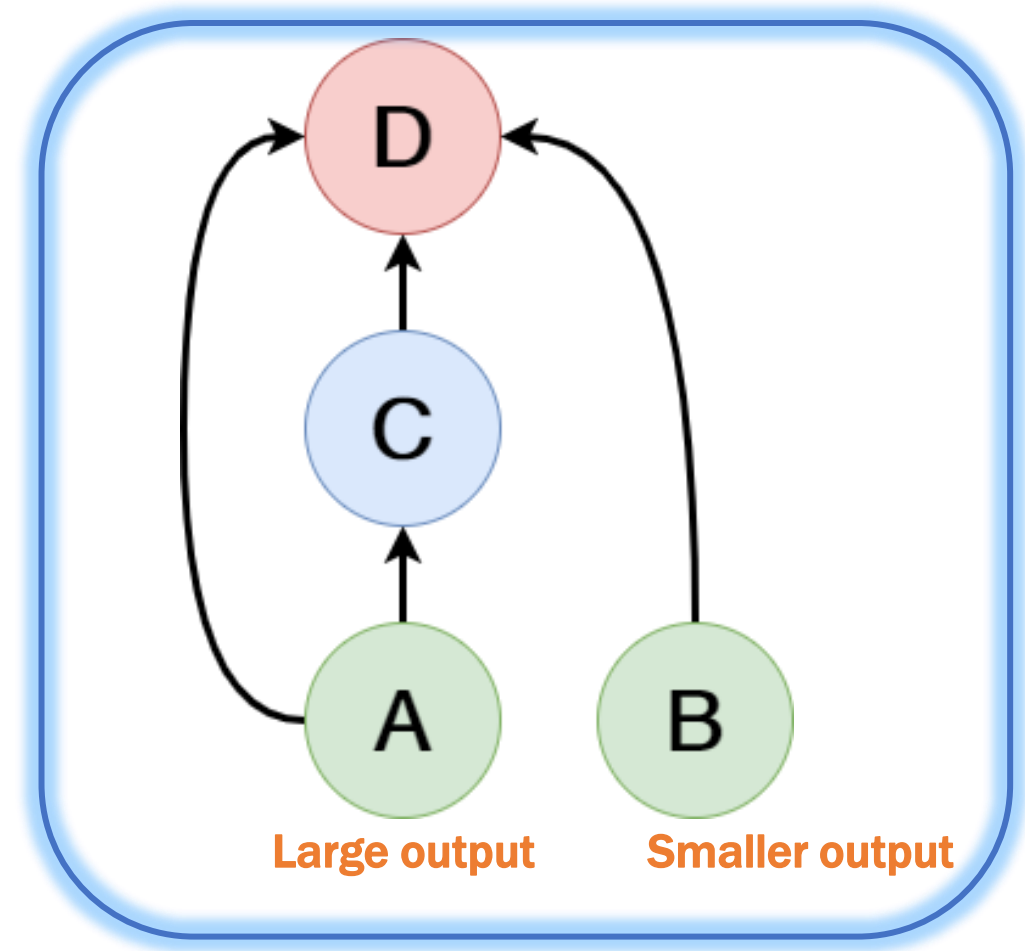
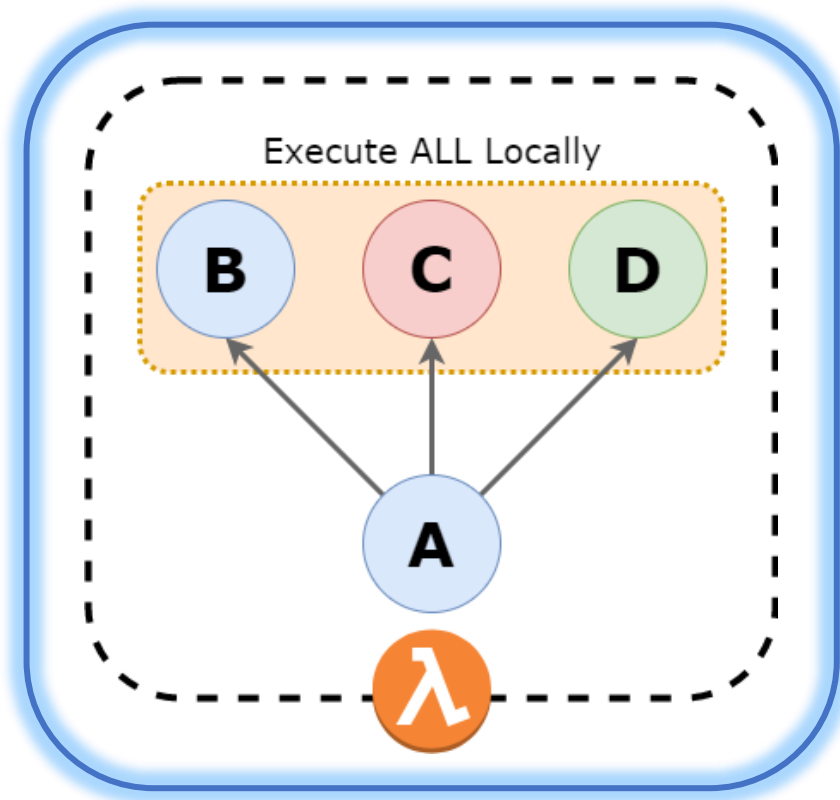
Eliminate intermediate data transfer by **executing downstream tasks locally**.

- **Delayed I/O**

Delay performing intermediate data transfer by waiting until downstream tasks are ready to execute. Then, perform **task clustering** on those tasks.



Task Clustering & Delayed I/O



Key Design Takeaways

- WUKONG addresses serverless computing's restricted, outbound-only network connectivity and low bandwidth by introducing **decentralized scheduling**, **delayed I/O**, and **task clustering**.
- These techniques **increase data locality**, greatly **reducing network I/O** performed during job execution.
- Less I/O → better performance → increased **cost effectiveness**.
- **Decentralized scheduling** also enables WUKONG to rapidly scale-out to **thousands of serverless Executors in seconds** by distributing the task of invoking Lambda functions across the existing workers.

WUKONG Evaluation

Implementation: Python 3, roughly 12,000 lines of code.

We compared **WUKONG** against **Dask distributed**, a serverful Python data analytics framework, and **numpywren**, a state-of-the-art serverless linear algebra framework.

Our goal – to answer the following two questions:

- (1) Can serverless provide competitive performance vs. serverful?
- (2) How does WUKONG perform compared to a related state-of-the-art serverless framework?

Experimental Setup

WUKONG

- **AWS Lambda**
 - 3GB Memory
 - Timeout set to seven minutes
- **AWS EC2**
 - Static Scheduler and MDS Proxy running on r5n.16xlarge VMs.
- **AWS Fargate**
 - 75 Fargate nodes, each configured with 30GB of RAM and 4 vCPU

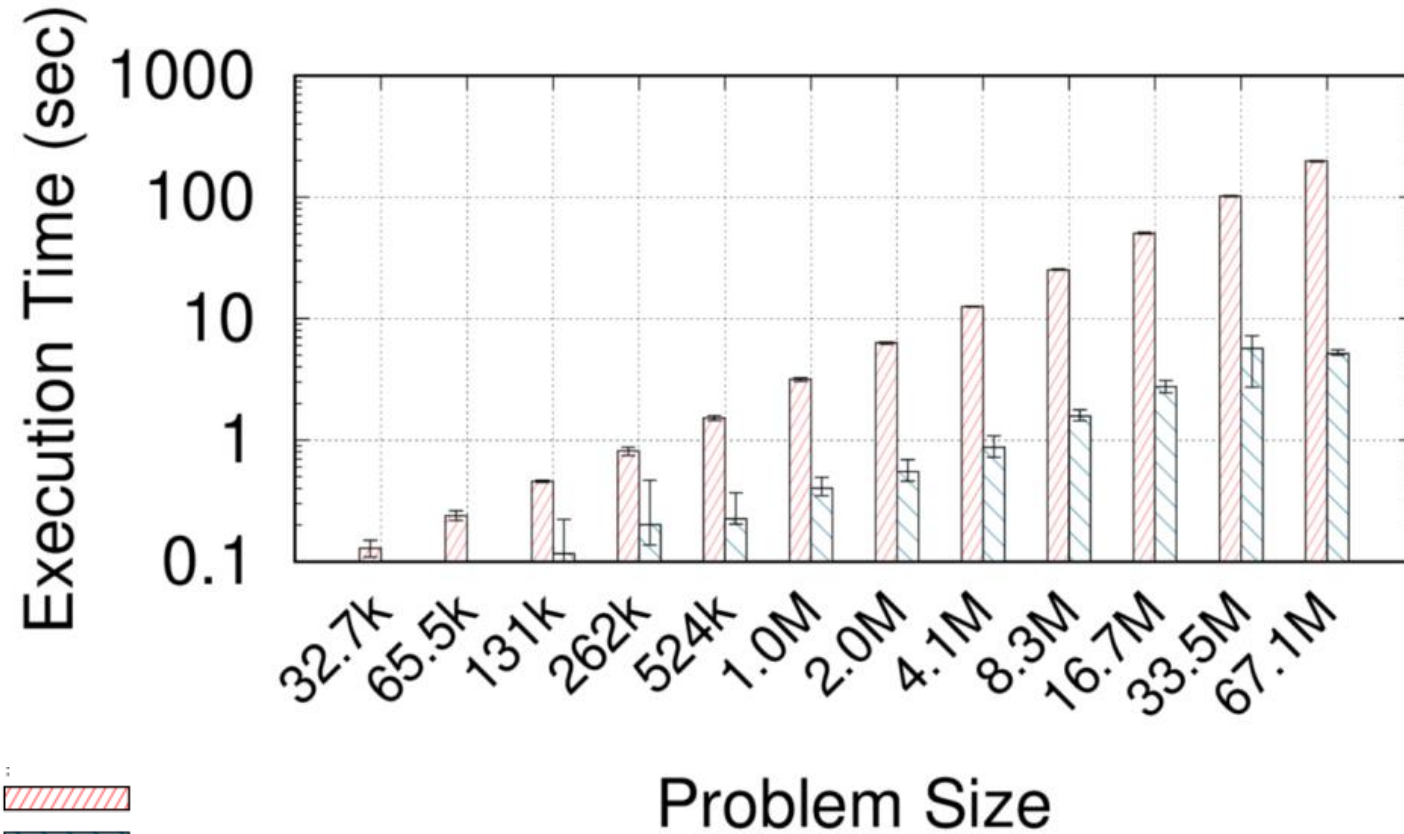
Dask distributed

- **1,000 2-Core Workers**
 - Ran on 125 c5.4xlarge VM's
- **125 16-Core Workers**
 - Ran on 125 c5.4xlarge VM's

Numpywren

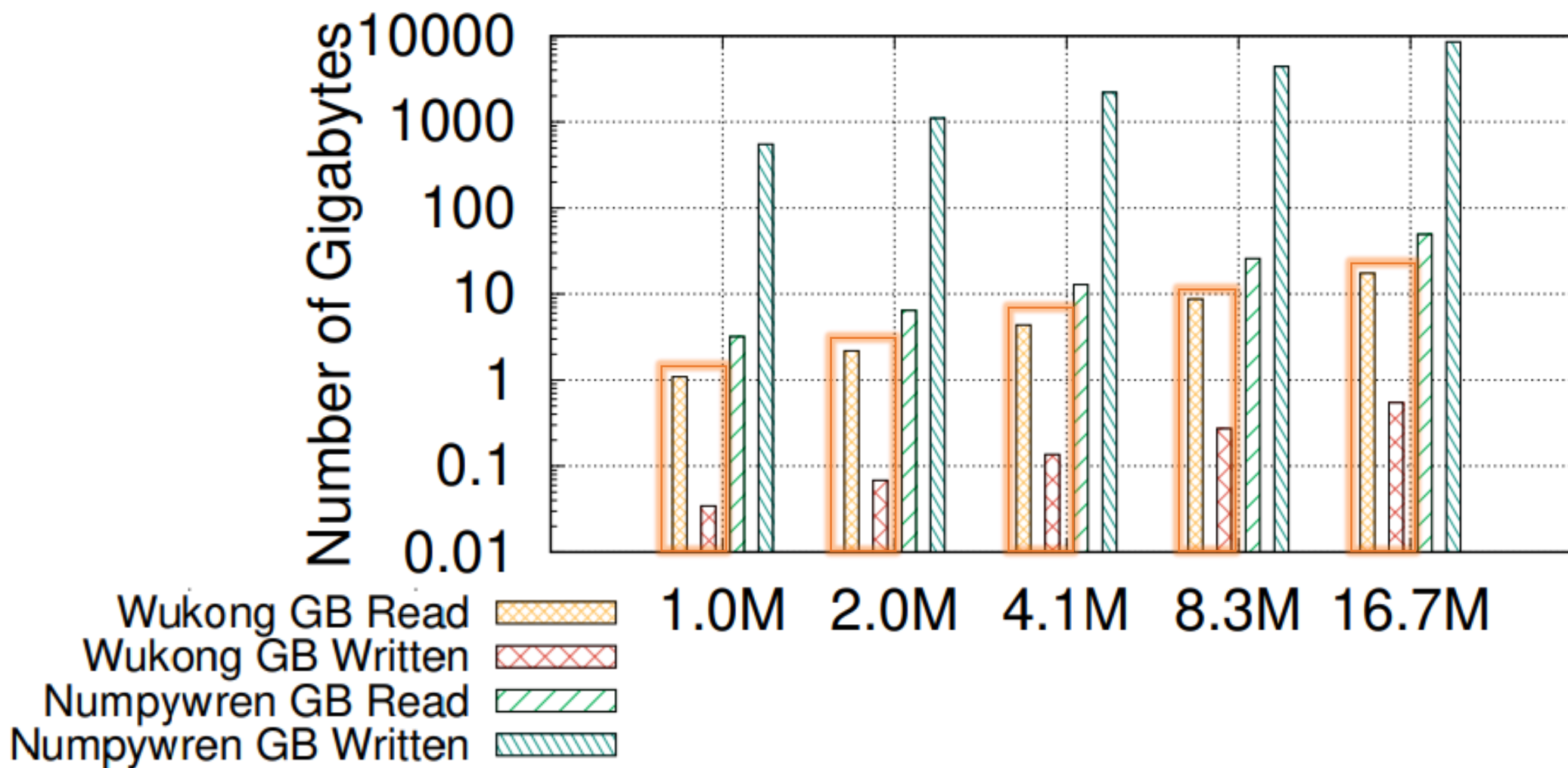
- **AWS EC2**
 - Scheduler on a r5n.16xlarge VM.
- **AWS Lambda**
 - 3GB of RAM; timeout set to 7 minutes.

Application: TSQR

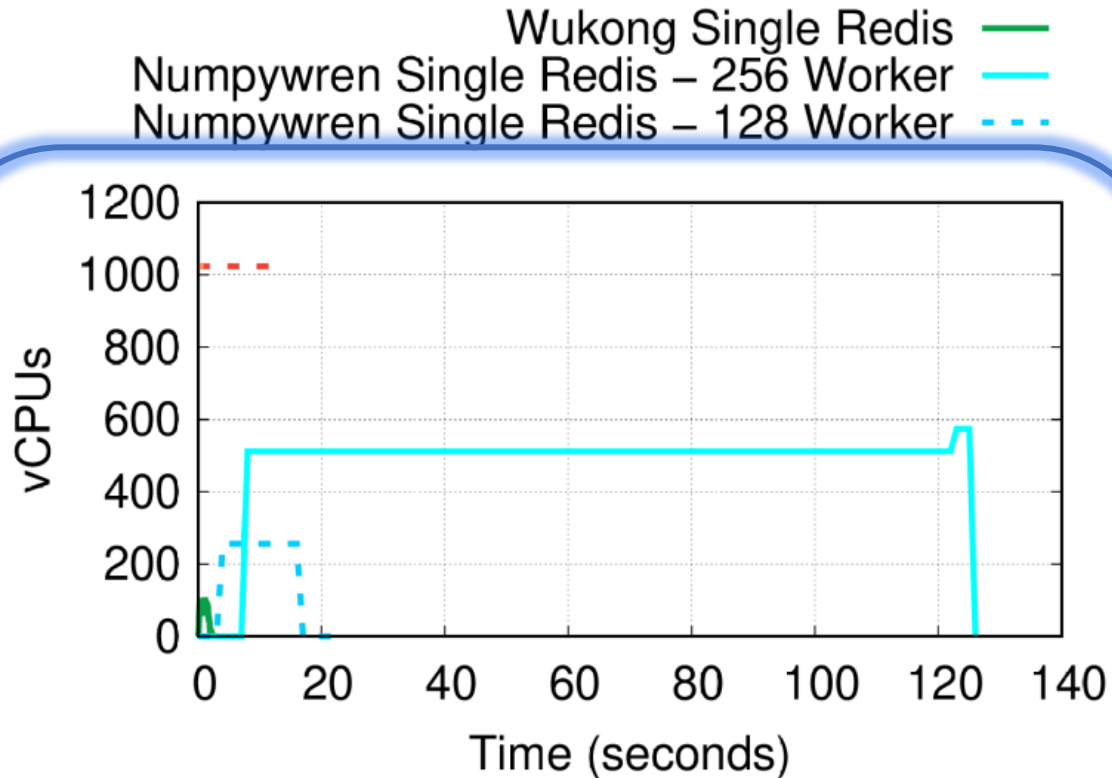


- Dask (1000 2C Workers)
- Dask (125 16C Workers)
- Numpywren S3
- Numpywren Single-Redis
- Wukong Multi-Redis
- Wukong Single-Redis

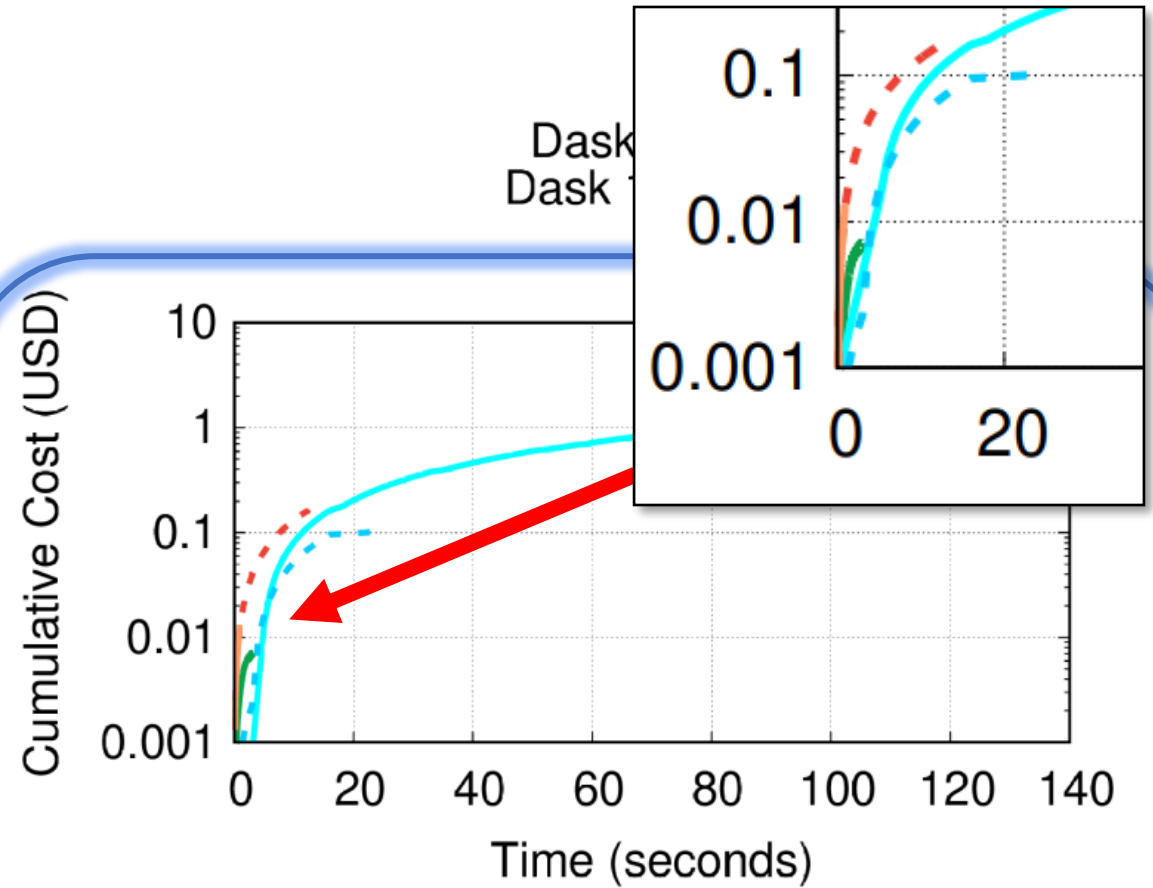
TSQR I/O Comparison



What about cost? (TSQR)

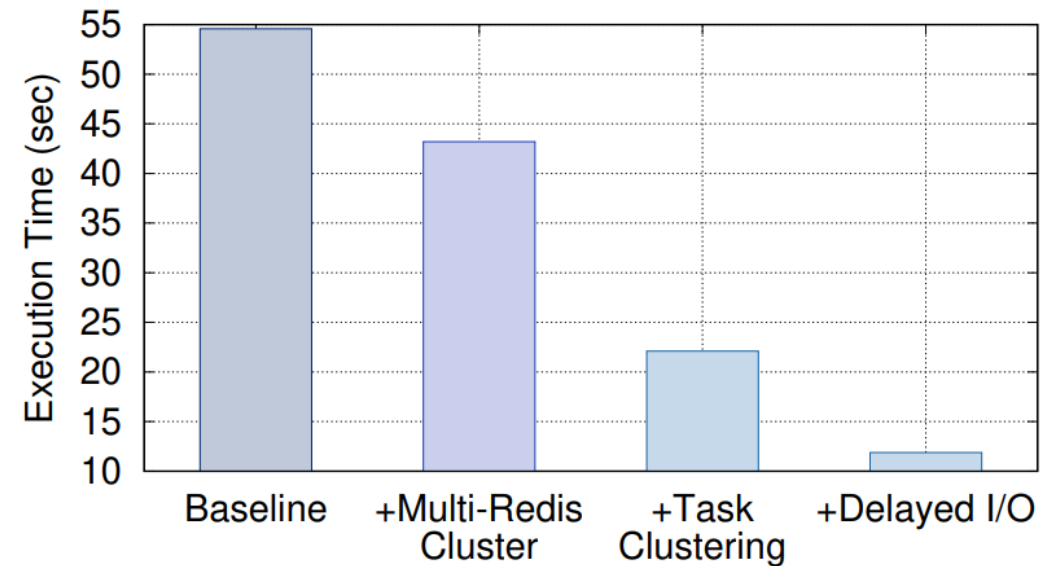
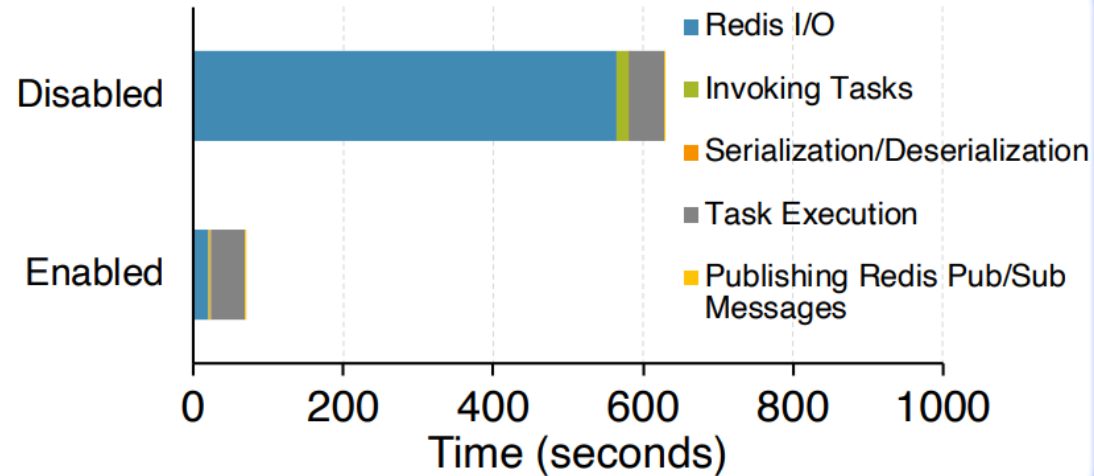


(a) CPU usage comparison.



(b) Cost comparison.

Does increased data locality *really* make a difference?



SVD 50k x 50k

Conclusion

- **WUKONG** is a scalable, locality-enhanced framework for serverless parallel computing.
- **WUKONG** can execute jobs up to **68x faster**, reduce network I/O by **multiple orders of magnitude**, and **achieve up to 92.96% tenant-side cost savings** compared to state of the art.
- **WUKONG** is **easy-to-use**, requiring **minimal configuration**, and provides **competitive performance** against comparably-priced serverful frameworks.

Thank you!

Source code available: <https://mason-leap-lab.github.io/Wukong/>

Contact:

benrcarver@gmail.com, or

bcarver2@gmu.edu



UNIVERSITY of
HOUSTON

What about cost? (SVD)

